

Lab 1 — Working Environment and a First End-to-End Pipeline

Machine Learning I · Tutorial

Rodrigo Grijalba (teaching assistant) · Alexander Quispe (instructor)

Wednesday, September 9, 2026 · 19:00–22:00

Goal of the session. Leave with a working Python environment, a copy of the course repository, and a complete supervised-learning pipeline you wrote yourself: data → split → fit → validate → report. Every number in Lecture 1's worked example gets recomputed on the way.

Materials

- `lab_01_student.ipynb` — the notebook you work in. Cells marked # TODO are yours.
- `lab_01_solution.ipynb` — released after the session.
- Sources: CS 155 Recitation 1 (Python) and Problem Set 1 (bias-variance, SGD).

1 Environment (20 min)

1.1 Option A — Google Colab

Open <https://colab.research.google.com>, upload `lab_01_student.ipynb`. Everything needed is preinstalled. Runtime → Change runtime type → CPU is enough for ML I.

1.2 Option B — local

```
python3 -m venv .venv && source .venv/bin/activate          # Windows: .venv\
Scripts\activate
pip install numpy pandas scikit-learn matplotlib jupyter
jupyter lab
```

1.3 The repository

```
git lfs install          # once; the reference PDFs are stored
with LFS
git clone https://github.com/alexanderquispe/enei-machine-learning-2026.git
cd enei-machine-learning-2026/ml1/labs/lab_01_pipeline
```

Pull before each session: `git pull`. You will never need to push.

2 numpy for Machine Learning (30 min)

Everything in this course is a matrix computation. Five things to be fluent in:

1. Shapes. `X.shape` is (N, p) ; `y.shape` is $(N,)$. A shape mismatch is the most common bug you will write this term.
2. Matrix product. `X.T @ X`, never `X.T * X` (that is elementwise).
3. Broadcasting. `X - X.mean(axis=0)` subtracts the column means from every row.
4. Solving, not inverting. `np.linalg.solve(A, b)` rather than `np.linalg.inv(A) @ b`. Same answer, better numerics, and it fails loudly on a singular `A`.
5. Randomness. `rng = np.random.default_rng(155)`; then `rng.normal`, `rng.permutation`. Seeded, so a result can be reproduced.

3 Lecture 1, Recomputed (40 min)

3.1 The worked example

Task 1 — normal equations by hand

With $x = (1, 2, 3, 4, 5)$ and $y = (2.2, 2.8, 4.5, 3.7, 5.5)$, build the design matrix $\mathbf{X}$ (column of ones first), form $\mathbf{X}^\top \mathbf{X}$ and $\mathbf{X}^\top \mathbf{y}$, and solve. You must obtain $\hat{\boldsymbol{\beta}} = (1.49, 0.75)^\top$ — Lecture 1, slide 15.

3.2 The hat matrix, numerically

Task 2 — four properties of $\mathbf{H} = \mathbf{X}(\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top$

Compute $\mathbf{H}$ (5×5) and verify, to floating-point precision:

1. $\mathbf{H}^\top = \mathbf{H}$ (symmetric)
2. $\mathbf{H}\mathbf{H} = \mathbf{H}$ (idempotent)
3. $\text{tr } \mathbf{H} = 2 = p + 1$ (degrees of freedom)
4. $\mathbf{X}^\top (\mathbf{y} - \mathbf{H}\mathbf{y}) = \mathbf{0}$ (residuals orthogonal to the columns)

Use `np.allclose`. Then check that the eigenvalues of $\mathbf{H}$ are $\{1, 1, 0, 0, 0\}$ and explain why (Lecture 1, slide 20).

3.3 Gradient descent

Task 3 — implement, then break it

Implement gradient descent for $L(\boldsymbol{\beta}) = \frac{1}{2N} \|\mathbf{y} - \mathbf{X}\boldsymbol{\beta}\|^2$, starting at $\boldsymbol{\beta} = \mathbf{0}$.

1. With $\eta = 0.05$ and 2000 iterations, confirm convergence to $(1.49, 0.75)$.
2. Compute $\lambda_{\max}(\mathbf{X}^\top \mathbf{X}/N)$ and the critical step $2/\lambda_{\max}$. Run with η slightly below and slightly above it. Plot L against iteration in both cases.
3. Standardize x (subtract mean, divide by standard deviation) and repeat. How does $\lambda_{\max}$ change? How many iterations does convergence take now?

The last item is the practical lesson of the section: scale your features. It is the cheapest conditioning fix there is.

4 A Real Pipeline with scikit-learn (50 min)

The dataset is `sklearn.datasets.load_diabetes`: $N = 442$ patients, $p = 10$ standardized co-variates, a quantitative disease-progression score as target. It ships with scikit-learn; no download.

4.1 The protocol

```
from sklearn.model_selection import train_test_split
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.2, random_state=155)
```

The test set is now sealed. It is opened once, at the very end, to report one number.

4.2 Baseline and first model

Task 4 — two numbers to beat

1. Baseline: predict $\hat{y}_{\text{train}}$ for everyone. Compute its MSE on the training set. This is the number any model must beat.
2. Fit `LinearRegression` on the training set. Report training MSE and R^2 . Verify that `model.coef_` equals what `np.linalg.solve` gives on the same data.

4.3 K -fold cross-validation, by hand

Task 5 — write `kfold_mse`

Write a function that takes $(\mathbf{X}, \mathbf{y}, K, \text{model})$, splits the rows into K folds with a seeded permutation, trains on $K - 1$ folds and evaluates on the held-out one, and returns the K validation MSEs. Compare its mean with `sklearn.model_selection.cross_val_score(..., scoring="neg_mean_squared_error")`. They should agree when both use the same fold assignment (`KFold(K, shuffle=True, random_state=155)`).

4.4 Model selection: the bias–variance picture, for real

Task 6 — polynomial degree

Using `PolynomialFeatures(degree=d)` for $d = 1, 2, 3$, compute training MSE and 5-fold CV MSE for each. Plot both against d . Training error must fall monotonically; CV error should rise by $d = 2$ or 3. Explain, in one sentence each, which term of the bias–variance decomposition (Lecture 1, slide 41) is responsible for each curve’s behaviour.

4.5 Learning curves

Task 7 — how much data is enough?

For training-set sizes $n \in \{20, 40, 80, 160, 320\}$, fit the linear model on the first n training rows and record training MSE and validation MSE (on a fixed 20% validation slice of the training set). Plot both against n . The gap between the curves is an estimate of the variance term; its closing rate tells you whether more data would help this model.

4.6 The one number

Only now: evaluate the chosen model (degree from Task 6) on $\mathbf{x}_{\text{te}}$. Report the test MSE. Do not go back and change anything afterwards.

5 Exercises (to submit before Lab 2)

Exercise 1. Implement mini-batch SGD for the least-squares loss with batch sizes $B \in \{1, 16, 442\}$ and a constant η . Plot the loss after each epoch for 30 epochs, all three on one axis. Which one reaches the lowest loss? Which one gets close fastest in wall-clock time? Explain both with Lecture 1, slides 35–36.

Exercise 2. Prove that for a random subset $\mathcal{B}$ of size B drawn without replacement, the mini-batch gradient $\frac{1}{B} \sum_{i \in \mathcal{B}} \nabla \ell_i(\boldsymbol{\beta})$ is an unbiased estimate of $\nabla L(\boldsymbol{\beta})$. Then compute its variance in terms of B , N , and $\text{Var}_i(\nabla \ell_i(\boldsymbol{\beta}))$, and check that it vanishes at $B = N$.

Exercise 3. On the diabetes data, add the $\binom{10}{2} = 45$ pairwise interaction features. Training R^2 goes up. Does 5-fold CV R^2 ? Report both, and state what the result says about $N = 442$ relative to $p = 55$.

Submission

A single notebook, all cells executed top to bottom without error (Kernel $\rightarrow$ Restart & Run All), uploaded to the course drive before Wednesday, September 16, 18:00.

Name it `lab01_lastname_firstname.ipynb`.